ASSIGNMENT 9

Advanced SQL Queries Using Joins and Subqueries

Online Shopping Management System

1. Aim

To generate useful online shopping reports by combining related tables with joins and subqueries.

2. Report 1 — customer orders (INNER JOIN)

```
SELECT o.OrderID, c.CustomerName, o.OrderDate, o.Status
FROM Orders o
INNER JOIN Customers c ON c.CustomerID = o.CustomerID
ORDER BY o.OrderID;
```

OrderID	CustomerName	OrderDate	Status
1	Arun Prabu	2026-10-01	Delivered
2	Rahul Kumar	2026-10-03	Shipped
3	Priya Sharma	2026-10-05	Pending
4	Arun Prabu	2026-10-07	Delivered
5	Sneha Reddy	2026-10-10	Cancelled
6	Karthik Raj	2026-10-12	Delivered

Explanation: INNER JOIN returns orders that have a matching customer record.

3. Report 2 — order line details (multiple joins)

```
SELECT o.OrderID, c.CustomerName, p.ProductName,
    od.Quantity, od.UnitPrice,
    od.Quantity * od.UnitPrice AS LineTotal
FROM Orders o
JOIN Customers c ON c.CustomerID = o.CustomerID
JOIN OrderDetails od ON od.OrderID = o.OrderID
JOIN Products p ON p.ProductID = od.ProductID
ORDER BY o.OrderID, od.OrderDetailID;
```

OrderID	CustomerName	ProductName	Quantity	UnitPrice	LineTotal
1	Arun Prabu	Laptop	1	55000.00	55000.00
1	Arun Prabu	Headphones	2	1500.00	3000.00
2	Rahul Kumar	Smartphone	1	25000.00	25000.00
2	Rahul Kumar	Keyboard	2	800.00	1600.00
3	Priya Sharma	Headphones	1	1500.00	1500.00
4	Arun Prabu	Office Chair	1	4500.00	4500.00
5	Sneha Reddy	Smartphone	1	25000.00	25000.00

Explanation: Joins four tables and calculates each order line as Quantity × UnitPrice.

4. Report 3 — order totals using GROUP BY

```
SELECT o.OrderID, c.CustomerName,
    SUM(od.Quantity * od.UnitPrice) AS OrderTotal
FROM Orders o
JOIN Customers c ON c.CustomerID = o.CustomerID
JOIN OrderDetails od ON od.OrderID = o.OrderID
GROUP BY o.OrderID, c.CustomerName
ORDER BY o.OrderID;
```

OrderID	CustomerName	OrderTotal
1	Arun Prabu	58000.00
2	Rahul Kumar	26600.00
3	Priya Sharma	1500.00
4	Arun Prabu	4500.00
5	Sneha Reddy	25000.00
6	Karthik Raj	800.00

Explanation: Aggregates order lines for each order. These totals are calculated from order details, not from the Payments table.

5. Report 4 — customers with no orders (LEFT JOIN)

```
SELECT c.CustomerID, c.CustomerName
FROM Customers c
LEFT JOIN Orders o ON o.CustomerID = c.CustomerID
WHERE o.OrderID IS NULL;
```

CustomerID	CustomerName

Explanation: LEFT JOIN keeps all customers, and the WHERE clause selects customers without a matching order.

6. Report 5 — products priced above average (subquery)

```
SELECT ProductName, Price
FROM Products
WHERE Price > (SELECT AVG(Price) FROM Products)
ORDER BY Price DESC;
```

ProductName	Price
Laptop	55000.00
Smartphone	25000.00

Explanation: The subquery calculates the average product price (17,360.00); the outer query returns products above that average.

7. Report 6 — customers with more than one order

```
SELECT CustomerName
FROM Customers
WHERE CustomerID IN (
   SELECT CustomerID
   FROM Orders
   GROUP BY CustomerID
   HAVING COUNT(*) > 1
);
```

CustomerName
Arun Prabu

Explanation: The subquery identifies customer IDs with more than one order. Arun Prabu has orders 1 and 4.

8. Report 7 — payment status summary

```
SELECT PaymentStatus, COUNT(*) AS PaymentCount,
     SUM(Amount) AS TotalAmount
FROM Payments
GROUP BY PaymentStatus
ORDER BY PaymentStatus;
```

PaymentStatus	PaymentCount	TotalAmount
Completed	4	89900.00
Pending	1	1500.00
Refunded	1	25000.00

Explanation: Groups payment records by status and sums their amounts. This includes pending and refunded records, so it is not the same as completed revenue.

9. Conclusion

Joins combine related records across tables, while subqueries filter records based on aggregate or related results. Run each query in MySQL Workbench and capture screenshots of the result grids for submission.